Day 1: Python, Pandas & Machine Learning Preprocessing

Complete Master Handbook & Beginner-Friendly Concept Guide

1. Core Pandas Foundations

• **Series:** A 1-Dimensional list with labels (index). Think of it as a single spreadsheet column.
• **DataFrame:** A 2-Dimensional table made of multiple Series sharing the same row index.
• **.loc vs .iloc:** .loc accesses data by **Label / Name** (e.g., df.loc[df['Age'] > 30]). .iloc accesses data by **Integer Position** (e.g., df.iloc[0:5, 0:2] for first 5 rows and 2 columns).

2. Data Cleaning in a Nutshell

• **Duplicates:** df.drop_duplicates(subset=['id']) removes repeat rows that falsely bias models.
• **Missing Values (NaN):** Don't blindly drop! Impute numerical columns with **Median** (resistant to outliers) and categorical columns with **Mode** or 'Unknown'.
• **Dirty Text & Numbers:** Strip spaces (.str.strip()) and remove currency/commas (.str.replace(r'[\$,]', '', regex=True)) before converting with pd.to_numeric(..., errors='coerce').

3. Data Manipulation & Aggregations

• **Filtering:** Wrap conditions in parentheses: df[(df['Salary'] > 75000) & (df['Age'] <= 35)].
• **GroupBy:** Summarize categories: df.groupby('Department')['Salary'].agg(['mean', 'median', 'count']).
• **Merging:** Combine tables like SQL joins: pd.merge(df, df_dept, on='Department', how='left').

4. Exploratory Data Analysis (EDA)

• **5-Number Summary:** df.describe() reveals count, mean, min, 25%, 50% (median), 75%, and max.
• **Correlation:** df.corr(numeric_only=True) measures linear dependence between -1 and +1.
• **Visuals:** Histograms (distribution shape), Boxplots (outliers), Countplots (class imbalance), Heatmaps (relationships).

Goal	Pandas Code
Inspect Shape & Nulls	df.shape; df.isnull().sum()
Drop Duplicates	df.drop_duplicates(subset=['Emp_ID'])
Impute Missing Median	df['Salary'] = df['Salary'].fillna(df['Salary'].median())
Filter Multiple Conditions	df[(df['Age'] > 30) & (df['Dept'] == 'IT')]
GroupBy Summary	df.groupby('Dept')['Salary'].mean()
Left Join Tables	pd.merge(df, budget_df, on='Dept', how='left')

5. Machine Learning Preprocessing Explained in Simple Terms

Why do we need this? A Machine Learning model is just a math formula (Prediction = W1*X1 + W2*X2 + b). Because it is pure math, it cannot understand words, gets confused by big numbers, and cheats if you give it exam answers in advance.

Concept 1: The Secret Answer (y) vs. The Clues (X)

Real-Life Analogy: The Detective & The Crime

• **Target (y):** The Mystery you want to solve (e.g., y = df['Left_Company']: Did the employee quit? 1 = Yes, 0 = No).

• **Features (X):** The Clues the detective uses to guess (Age, Department, Salary, Rating, Experience).

• **Why drop 'Emp_ID' and 'Name'?** If a detective says *'Employee #104 named David quit, so every future person named David will quit!'*, that is terrible logic! Names and IDs are random unique labels. Keeping them causes the model to memorize badges rather than learning real human patterns.

```
Code: y = df['Left_Company']
X = df.drop(columns=['Emp_ID', 'Name', 'Dept_Head', 'Left_Company'])
```

Concept 2: Turning Words into Numbers (One-Hot Encoding)

Why can't we just write IT = 1, HR = 2, Sales = 3?

Because computers do math! If IT=1 and Sales=3, the computer thinks: *'Sales is 3 times bigger than IT'* and *'IT (1) + HR (2) = Sales (3)'*! That is completely absurd.

The Solution (Light Switches): We create a separate 1 or 0 column for each department:

• If Alice is in IT → Dept_IT = 1, Dept_HR = 0, Dept_Sales = 0.

• drop_first=True: If both Dept_IT=0 and Dept_HR=0, the model already knows they are in Sales! Dropping 1 redundant column prevents mathematical confusion (the dummy variable trap).

```
Code: X_encoded = pd.get_dummies(X, columns=['Department'], drop_first=True)
```

Concept 3: Train vs. Test Split (Homework vs. Surprise Exam)

Real-Life Analogy: The Student & The Final Exam

If a teacher gives you the exact exam questions to practice at home yesterday, and you score 100% today, you didn't learn—you just memorized answers!

• **Training Set (75%):** The practice textbook exercises the model is allowed to study.

• **Testing Set (25%):** The surprise exam. The model has never seen these rows before. This is how we prove the model actually works in the real world.

• **stratify=y:** Ensures the test exam has the exact same proportion of quitters (30%) and stayers (70%) as real life.

```
Code: X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.25,
random_state=42, stratify=y)
```

Concept 4: Feature Scaling (The Giant vs. The Ant)

The Problem: Look at two features for an employee:

- Salary = **$90,000** (A massive giant)
- Rating = **4.5** (A tiny ant)

When the model calculates W1 * 90000 + W2 * 4.5, the $90,000 completely drowns out the 4.5! The model will ignore Rating entirely, even though performance rating is often the #1 reason people leave!

The Fix (StandardScaler): We convert all numbers into how many steps they are away from their average:

- Average Salary becomes **0**. A high salary becomes **+1.5**. A low salary becomes **-1.2**.
- Average Rating becomes **0**. A high rating becomes **+1.4**.

Now both features compete on a level, fair playing field!

Golden Rule: fit_transform() ONLY on train data. transform() on test data. Never let the scaler look at the test exam (prevents data leakage)!

Code:

```
scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols]) # Learn average from homework & scale
X_test[num_cols] = scaler.transform(X_test[num_cols]) # Use homework average on exam (NO PEEKING!)
```